Automatic Procedure Following Evaluation Using Petri Net-Based Workflows

Víctor Rodríguez-Fernández, Antonio Gonzalez-Pardo, *Member, IEEE*, and David Camacho, *Member, IEEE*

***Abstract*—An operating procedure (OP), also known as checklist or action plan, is a list of actions or criteria arranged in a systematic way, commonly used in areas such as aviation or healthcare to ensure the success of critical tasks and to help decrease human errors. In these areas, operators are hardly trained to follow the OP carefully, but the evaluation of how they are following, it is usually performed manually by an expert instructor. Automating this evaluation process would lead to an objective and scalable analysis of the operator performance, which is extremely important in areas where the number of operators to evaluate is high. This problem, which can be referred as automatic procedure following evaluation, needs of new techniques and formalizations due to current conformance checking methods do not fit well with some aspects of a OP. In this paper, OPs are modeled as Petri Net-based workflows, and interact with the data log of the system to allow an automatic evaluation of the progress and time spent following the OP. In order to illustrate the contributions of this paper, a case study is carried out designing and modeling an emergency OP for an unmanned aircraft system, and evaluating the proposed approach with a battery of tests.**

***Index Terms*—Checklist, conformance checking, operating procedure (OP), Petri Net (PN), procedure following evaluation (PFE), workflow.**

I. Introduction

THE number of human-dependant critical tasks is increasing every day in a large number of works. These tasks are considered critical because any failure during their execution could produce important consequences such as human life loss or high monetary loss. To reduce the risk involved in them, a step-by-step guiding tool [called operating procedure (OP)] is usually given, describing the different checks and actions that the person responsible for this task, namely the operator, must perform in order to solve it successfully. The concept of an OP is also known as a *checklist* in many domain fields, although it can be referred to as *action checklists* or *emergency OPs*.

There is a plethora of applications that use OPs to define the different steps required to solve a critical task. The most extended area of application is aviation, where pilots act in many high-risk situations such as takeoff emergencies, ejection procedures, landing emergencies, or fuel system failures among others [1]. OPs are also used in medicine and healthcare, in order to provide surgical care [2], or to define how to transport the different patients [3].

Due to the critical facet of the just mentioned tasks, operators are trained in order to get them used to follow OPs strictly. In this training, an instructor is in charge of controlling that the operator is correctly following the steps described in the OP. This is what we will, henceforth, call procedure following evaluation (PFE). Nowadays, in most of the applications involving OPs, the PFE is performed manually, i.e., the instructor must verify that an operator is following all the steps described in the OPs [4]. This manual supervision is not appropriate if the number of operators increases because instructors are not able to inspect whether each trainee is correctly performing all the steps described in the OP. For this reason, the development of systems that automatically evaluate and analyze how operators follow an operating procedure (OP) is highly important, not only for the scalability of the training phase, but also to extract measures about the performance of the operators.

According to the literature, the family of techniques focused on comparing process models with data from the same process is known as *conformance checking*. In recent years, some methods such as replay [5] or behavioral alignment [6] advocate for the use of Petri Net-based Workflows, or Workflow Nets (WF-Nets), to check the conformance between a business process model and an event log. However, these techniques are not suitable for PFE for two main reasons: On the one hand, the business process models are *event based*, which imply that the condition to fulfil an activity in the WF-Net consists just in checking the presence of a single event in the log. Instead, OPs are *state-based* processes, and some procedural steps must be checked according to flexible and complex conditions involving the state of one or many variables in the data log during a certain period of time, for example, the step "Supervise that the altitude of the vehicle is below 10 000 m for 1 min." On the other hand, the concept of time is secondary in most of the conformance checking methods, and here it is a critical factor not only for

Manuscript received April 24, 2017; revised July 14, 2017, September 30, 2017, and November 17, 2017; accepted November 27, 2017. Date of publication December 4, 2017; date of current version June 1, 2018. This work has been supported by the next research projects: Airbus Defence & Space (FUAM-076914 and FUAM-076915), EphemeCH (TIN2014-56494-C4-4-P) and DeepBio (TIN2017-85727-C4-3-P) Spanish Ministry of Economy and Competitivity, CIBERDINE S2013/ICE-3095, both under the European Regional Development Fund FEDER, and RiskTrack (JUST-2015-JCOO-AG-723180). Paper no. TII-17-0828. *(Corresponding author: Víctor Rodríguez-Fernández.)*

The authors are with the Departamento de Informática, Universidad Autónoma de Madrid, Madrid 28049, Spain (e-mail: victor.rodriguezf@uam.es; antonio.gonzalez@uam.es; david.camacho@uam.es).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TII.2017.2779177

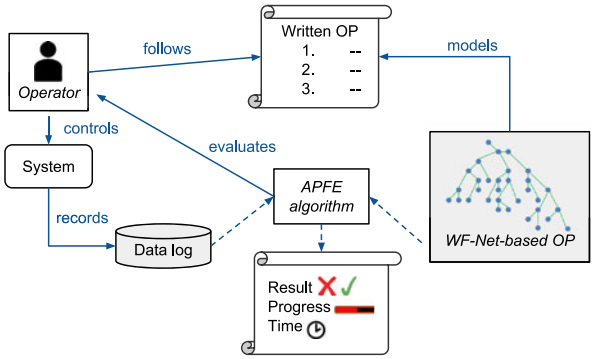

Fig. 1. General concept of the proposed approach.

evaluating the time spent to complete an OP, but also to define *a priori* the maximum step duration allowed for each procedural step.

Thus, this paper extends the use of WF-Nets with new formalizations and methodologies to model OPs and execute conformance checking (or PFE) on them. The problem of automating the PFE is solved by expressing the contents of an OP as a formal model in which we can introduce the data log resulted from the response of the operators in the system, in order to analyze the progress achieved and time they spent on the process (see Fig. 1). The contributions of this paper can be summarized as follows:

1) We formalize and generalize the concepts and relations involved in an OP.
2) We describe a method to model OPs using (WF-Nets), both, the static and dynamic components of the model are adapted to the concepts and relations involved in the OP. We have selected Petri Nets (PNs) because they provide a formal mathematical framework that allows us to model both concurrent and sequential steps, which is extremely useful in cases where the operator is asked to perform more than one action at the same time.
3) We introduce an automatic procedure following evaluation (APFE) algorithm which uses the WF-Net-based OP to automatically evaluate the response of an operator following a specific OP. This evaluation is not performed in real time, but it works taking into account the system log that contains all the timestamped data resulted from the operator response.
4) In order to illustrate the ideas mentioned above, we develop a test case by using a realistic OP belonging to the field of unmanned aerial vehicle (UAV) operations.
5) A comparison study is carried out to remark the main advantages of this approach over previous Procedure Following Evaluation (PFE) approaches.

The rest of the paper is structured as follows: Section II provides the basic backgrounds on PNs, Workflow Nets, and OPs needed to contextualize the current paper. In Section III, the definition of the proposed modeling strategy is described, and Section IV presents the APFE algorithm based on an OP model. Then, Section V shows the case study of this paper, and in Section VI, we focus on comparing the proposed approach with the related work. Finally, the conclusions and future work can be found in Section VII.

II. Backgrounds

A. OPs and Checklists

A checklist, or OP is typically a list of action items or criteria arranged in a systematic manner, allowing a user to record the presence/absence of the individual items listed to ensure that all are considered or completed. Their efficacy as a cognitive aid lies in their ability to "chunk" item-specific information in an organized fashion, and in the fact that these lists of instructions are often better understood than information in paragraph format. As a consequence, the use of checklists usually results in an improvement of performance and memory recall, and a decrease in the human error rate [7].

Checklists have been widely and consistently used, at least in the following two different areas.

1) *Aviation*: Checklists constitute a critical part of the flight protocol. Two types of checklists are usually defined in this field: a) *Normal checklists*, which guide regular flight practices such as takeoff, landing, etc., b) *Emergency checklists*, used to guide the correction of alert situations, such as landing emergencies, fuel system failures, etc.
2) *Healthcare*: Checklists have been demonstrated to be effective in high-intensity fields of medicine, such as trauma and anesthesiology [8].

Reijers *et al.* [9] discuss about the philosophy of design of a checklist and propose a checklist classification in terms of several characteristics.

1) *The device*: Although the most common device is a piece of paper, we can also find scroll, mechanical, vocal, and electronical checklists [10].
2) *The Method*: There are two predominant methods of conducting a checklist: a) *Do-list*: The checklist follows a step-by-step "cookbook" approach. b) *Challenge-response*: The checklist is a backup procedure that is used after a specific operation to verify that all the items listed on the checklist have been correctly accomplished.
3) *The Items on the List*: The type and the amount of items included in a checklist is a cardinal question in the checklist philosophy.
4) *Level of Automation*: It discusses whether to include in the checklist some steps that could be automatically verified by the system or not.

B. Petri Nets

A classical (or low-level) PN is a directed bipartite graph with two node types called *places* and *transitions*, and a set of directed *arcs* that connect nodes of different types. Connections between nodes of the same type (i.e., *place* to *place* and *transition* to *transition*) are not allowed.

At any time of a process, a *place* in a PN can contain zero or more *tokens*, usually drawn as dots inside the place. The distribution of tokens over places is often referred as *marking*, and represents the status of the net at any (discrete) time.

Formally, a PN can be described as a quintuple (P, T, I, O, M), where the following statements hold.

1) $P = \{p_1, \ldots, p_{n_p}\}$ is a finite set of n_p places.
2) $T = \{t_1, \ldots, p_{n_t}\}$ is a finite set of n_t transitions.

3) $I \subseteq (P \times T)$ is a set of *input arcs*, which connect places to transitions. A place p is called an *input place* of a transition t iff there exists an input arc from p to t.
4) $O \subseteq (T \times P)$ is a set of *output arcs*, which connect transitions to places. A place p is called an *output place* of a transition t iff there exists an output arc from t to p.
5) $M : P \rightarrow \mathbb{N}$ is the *marking* of the net in a specific moment. We can represent a marking as follows: $1p_1 + 3p_2 + 5p_4$ to show a state that has one token in place p_1, 3 tokens in p_2, 5 in p_4, and 0 tokens in the rest of the places. Furthermore, $M(p)$ represents the number of tokens of place p in marking M.

Transitions are the active components of a PN, that are used to change the position and number of tokens during the execution of the net. The dynamics of a classical PN follow some *execution rules*.

1) A transition t is said to be *enabled* in a marking M iff each input place of the transition contains at least one token, i.e., iff $M(p) > 0, \forall p \in \bullet t$, where $\bullet t$ refers to the set of input places of the transition t.
2) When a transition t is enabled, it may *fire*, *consuming* one token from each input place $p_{in} \in \bullet t$ and *producing* one token in each output place $p_{out} \in t\bullet$.

PNs have been widely used in multiple fields as a tool for describing and analyzing discrete-event dynamic systems [11], [12], due to the combination they offer of a great expressive power, legibility, and mathematical formality.

However, PNs describing real processes tend to be complex and extremely large, so it is not possible to use the standard PN to model complex data or time-dependent problems. For that reason, many extensions of a classical PN have been proposed, known as *high-level PNs*. In this paper, we are focused on a type of PNs where the tokens contain a data value, usually known as *token color* [13]. Examples of this type of PNs are *Colored PNs*, *E-nets*, or *First Input First Output (FIFO-) nets*, among others. Transitions describe the relation between the colors of the input tokens and the colors of the output tokens. Also, a transition can establish "preconditions" (*guards*) to enable/disable itself depending on the color of the input tokens.

C. Workflow Management (WfM) and WF-Nets

WfM includes methods, techniques, and tools to support the design, enactment, management, and analysis of workflows [14]. Workflows are *case based*, i.e., every piece of work is executed for a specific *case*. Cases are handled by executing *tasks* in a specific order. Each task has "pre-" and "post-" conditions, and they are described as TRUE/FALSE expressions.

It is important to use an established framework for modeling and analyzing workflow processes. The benefits of using PNs as a framework for the specification and modeling of workflows are the following.

1) The *formal* semantics.
2) The availability of many *analysis* techniques.
3) The fact that PNs are state-based rather than event based.

Modeling a workflow process in terms of a PN is rather straightforward: *tasks* are modeled by *transitions*, *conditions* are modeled by *places,* and *cases* are modeled by tokens. The application of PNs to WfM results in a class of PNs named WF-Nets [15], which satisfies the following.

1) It has one initial place (i), where the process starts, and one output end place (e), where the process ends.
2) Every task transition and place should be located on a path from the initial place to the end place. That means that if we virtually extended the PN adding a transition which connects place e with i, then the resulting PN should be strongly connected.

A WF-Net can be used to describe the routing of cases throughout the workflow, building blocks such as the AND-split, AND-join, OR-split, and OR-join in order to model sequential, conditional, parallel, and iterative routing [5].

III. Modeling an OP as a WF-Net

In this section, we provide a complete description of how to adapt the elements of a WF-Net to model the concepts and relationships present in a written OP. The resulting model, named as WF-Net-based operating procedure (OP), will "interact" with the data log in of the system, in order to evaluate the operator response to a critical task (see Fig. 1). To achieve this, first of all the concepts and relationships of an OP must be described, and then modeled in terms of WF-Net elements. Finally, the dynamics of the model must be defined in order to make it appropriate for evaluating the operator response.

A. Concepts and Relations for OPs

In this paper, we will refer to the term "Operating Procedure" as a way to gather different step-by-step guiding tools, such as *checklists*, *action checklists*, *Emergency OPs*, *action plans*, etc.

Here a description of the main concepts and relations for OPs is given. This description has been defined based on works from different fields where procedures and checklists have been proven to be successful in real operational environments. This includes the use of the WHO Surgical Safety Checklist, the study of nuclear OPs as the loss of coolant accident procedure [16], and finally, the analysis of OPs from the field of aviation, such as the Airbus manual for emergencies in UAV missions.[1]

Basically, an OP comprises an ordered sequence of *procedural steps*, each of them describing a subgoal that the operator must accomplish to successfully complete the whole procedure. A procedural step may be either atomic or a container of a sequence of substeps, which must be all fulfilled in order to consider the container step has been completed. Additionally, a procedural step contains an attribute, namely μ, indicating the maximum step duration, which in some cases can be provided *a priori* by a domain expert (the process modeler), and in other cases, it can be computed in terms of a step complexity measure.

Based on the type of response expected from the operator in a procedural step, each atomic procedural step can be classified according to three groups.

[1]References and specific details from these procedures have not been included due to a confidential agreement with Airbus Defence and Space Company.

1) *Action*: Represents a step in which the operator is asked to do something that will alter in some way the status of the system. Actions are usually mandatory for the completion of the OP and consume a specific amount of time. An example of an Action Step might be: *"Command altitude change to 10 000 ft."*
2) *Check*: A check is a step in which the operator is asked to confirm/refuse that a specific property inside the operational environment is fulfilled. Usually, the process of checking a property is performed visually. In this paper, we consider that checks are *instantaneous* ($\mu = 0$), which means that the time employed by the operator is irrelevant. Examples of a check step would be: *"Confirm the remaining fuel is higher than 50 L"* or conditional sentences like *"If the remaining fuel is higher than 50 L..."*).
3) *Supervision*: There are some procedural steps that ask the operator to supervise the status of some aspects of the system for a time period (e.g., *"Verify that the UAV aborts autonomously and lands"*). Unlike check steps, the process of supervising is not instantaneous, but persistent during a time interval ($\mu \neq 0$). A conditional statements of an OP involving time, such as "While ..." and "When ..." can be modeled as a supervision step.

B. Implementing the Concepts of the OP Using WF-Net Elements

Once we have formalized the different concepts and relations comprising an OP, it is possible to express each of those concepts in terms of classical PN elements, following common workflow design patterns.

As in every PN-based workflow, a WF-Net-based OP starts with an unique initial place and finish with a unique end place. Between them, consecutive procedural steps follow one another, linked by arcs. However, unlike for common WF-Nets, there is not a direct mapping between atomic PN elements and procedural steps. Instead, the concepts of an OP become meaningful only in terms of blocks. The presence of cycles is allowed in a WF-Net-based OP, as long as it corresponds to a natural logic contained in the written procedure that is being translated (e.g., "If the conditions of step 3 are not fulfilled, go back to step 1").

Fig. 2 shows the definition of the different type of procedural steps described in terms of blocks of places, transitions, and arcs. Actions are the simplest steps to model, only a sequential combination *"Place-Arc-Transition"* is needed. This is because when an OP requires the action of an operator, there is no other way to follow than waiting for the operator to execute that action successfully. On the other hand, *Check* steps and *Supervision* steps always offer two different ways to follow the execution of the OP, depending on whether the condition to check or supervise is *fulfilled* or not. This fact is modeled as a place connected to two mutually exclusive transitions, which can be seen as an XOR-*Split* routing block.

Regarding container steps, we distinguish between *concurrent* and *sequential* containers, depending on whether the OP allows operators to perform those substeps in parallel or not.

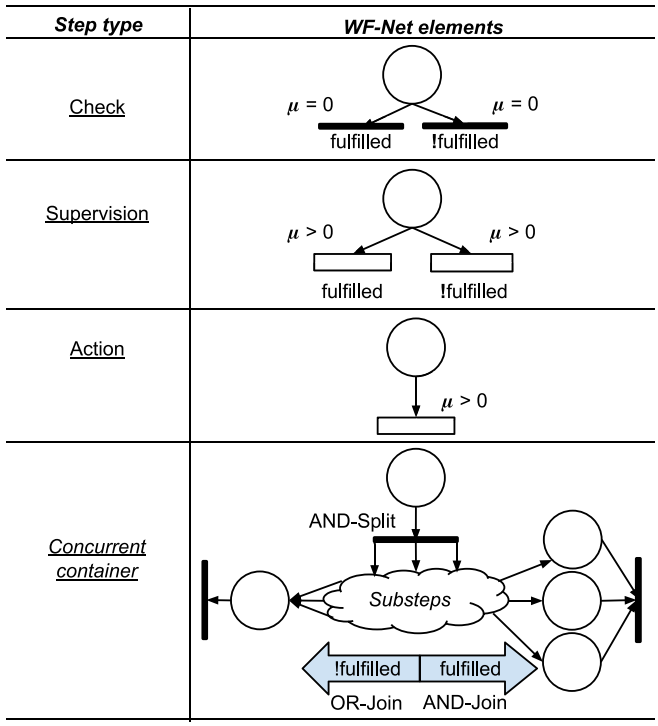

Fig. 2. Workflow process definition for the different type of procedural steps defined for any OP.

In case of having substeps concurrency, we surround them with an AND-*split* routing block at the beginning and an AND-*Join* at the end, to ensure that every substep is analyzed and that *Step N+1* cannot start until every substep from (*Step N*) has finished. In addition, if there are check or supervision steps inside the concurrent container, we must link, on the one hand, the `fulfilled` transitions of each of them to the aforementioned AND-*Join* block, and on the other hand the `!fulfilled` transitions must be directed to an OR-*Join block* that gathers every possible nonfulfillment of the concurrent container (see Fig. 5 as an example).

C. Dynamic Behavior of the WF-Net-Based OP

In order to understand how the execution of an OP is implemented inside a PN in this paper, tokens, markings, and firing policies in the net must be specified. In a WF-Net-based OP, the support for the movement of tokens comes from the *data log* of the operation.

Definition 1: A *data log* $\Delta = (V, U, R)$ consists of the following.

1) A set V of variables.
2) A function U that defines the values admissible for each variable, i.e., $U(v)$ is the domain of variable v for each $v \in V$. In this context, $D = \bigcup_{v \in V} U(v)$ represents the set of all possible data values of any variable.
3) A function R, that records the values of each variable over time, i.e., $R : [0, \infty) \times V \mapsto D \cup \{\perp\}$, with $R(x, v) \in U(v) \cup \{\perp\}$. [2]

[2]If a variable is not given a value, we use the special symbol $\perp$.

Note that R is defined as a partial function, due to the values of each variable are recorded only at specific timestamps. Thus, the domain of R, $dom(R) \subset [0,\infty) \times V$ specify the times and variables for which the data log contains a recorded value. Either the records are created synchronously (with a preset record rate) or asynchronously (whenever a variable changes), the key issue in the above definition is that it allows us to track over time the variables governing the operation to be analyzed. This definition of a data log comes from both the ideas presented in [17], where authors convert unstructured text into a structured log that shows the logged variables in (name, value) pairs, and the notation used for *Data PNs* in [18], to which we have added the notions of temporal dependency in the values of variables.

The records contained in a data log can be seen as a set of *log entries* $\langle x, v, R(x,v)\rangle \in E$, where $E = [0,\infty) \times V \times D$. In order to navigate through the log entries continuously over time, we introduce the concepts of last record time and log state, which represents the closest past record of a variable at any moment of the process.

Definition 2: Given a data log $\Delta = (V, U, R)$, the *log state* is a function that extracts, for any timestamp $x \in [0,\infty)$ and any variable $v \in V$, the log entry that contains the closest past record of that variable in the log, i.e.,

$$S^{\Delta} : [0,\infty) \times V \to E$$
$$S^{\Delta}(x,v) = \langle \text{CPRT}(x,v), v, R(\text{CPRT}(x,v), v)\rangle$$

where CPRT is the closest past record time, defined as

$$\text{CPRT} : [0,\infty) \times V \to [0,\infty)$$
$$\text{CPRT}(x,v) = \max_{\langle x', v'\rangle \in dom(R) | v' = v \wedge x' \leq x} (x').$$

Based on this, we introduce the set $S^{\Delta}_{x} = \bigcup_{v \in V} S^{\Delta}(x,v)$ which gathers the log entries of every variable in the data log at a given time. In other words, it represent the state of the process up to time x. In the same way, $S^{\Delta}_{[x,y]} = \left\{ S^{\Delta}(x', v) \mid x \leq x' \leq y, v \in V \right\}$ gathers all the log entries between x and y.

A token conceptually represents the *operator response* facing procedural steps. It is defined by two attributes.

1) *Position (p)*: Represents the place on the net where the token is located.
2) *Timestamp (x)*: It specifies the "age" of the token. This timestamp will be continuously changing by the dynamics of the net.

We can use the tuple $\langle p, x\rangle \in P \times [0,\infty)$ to denote a token in place p with timestamp x. A marking conceptually represents the current progress of the operator through the OP. Inside a marking, tokens are organized in places in form of unordered sets, so if two tokens that enable a transition share the same place, the decision of using one over the other is chosen randomly. In order to better handle the tokens in this paper, the formal definition of a marking is modified with respect to the one of classical PNs (see Section II). Let $K = P \times [0,\infty)$ be the space of tokens, a marking is defined as a function $M : P \to K^*$ that assigns a set of tokens to each place. [3] Furthermore, given a transition $t \in T$, $M(\bullet t) = \bigcup_{p \in \bullet t} M(p)$ refers to all the input tokens of t, and $M(t\bullet)$, defined analogously, to all the output tokens.

Transitions are the active components of the net, since they cause changes in the net marking when they are fired. In terms of the dynamics of the OP, they are responsible for the following.

1) Deciding whether a procedural step has been performed correctly or not.
2) Moving the token(s) in case that the procedural step has been performed.
3) Increase the timestamp of the token to the exact moment that the operator completed the step.

To formalize the firing policies of a WF-Net-based OP we define, for each transition $t \in T$, a control function, namely guard condition (GC), that is used to specify, in terms of the data log, whether the procedural step associated to the transition has been fulfilled or not.

Definition 3: Given a data log $\Delta = (V, U, R)$ and let E^* be the set of all possible finite sets of log entries $\langle x, v, d\rangle$ such that $v \in V, d \in U(v) \subset D$. A GC is a function $GC : E^* \to \{\texttt{true}, \texttt{false}\}$ that decides whether a set of log entries are admissible to fulfil a condition or not.

By including temporal information into the definition of the guards (recall that a log entry contains a timestamp), we allow the creation of complex and flexible conditions that could not be expressed via event log-based data guards [18]. This is especially important for expressing the statements associated to supervision steps in an OP (e.g., "Supervise that the altitude of the vehicle is below 10 000 m for 30 s"), due to they usually involve the verification of the state of some variables over time, rather than the occurrence of an event.

Based on the above definition, we can create a function, namely the time of completion (ToC), that receives an input token from an enabled transition t, and returns the earliest moment, after the timestamp of the token, when the GC of t is fulfilled, i.e., when the associated procedural step was completed.

Definition 4: Let $\Delta = (V, U, R)$ be a data log. Let x_{in} be the timestamp of an input token at a given transition $t \in T$. Let $\mu(t)$ be the maximum step duration of the procedural step to which the transition belongs. The ToC is denoted as

$$ToC : T \times [0,\infty) \to [0,\infty)$$
$$ToC(t, x_{\text{in}})$$
$$= \min \left\{ x \in [x_{\text{in}}, x_{\text{in}} + \mu(t)] \mid GC_t(S^{\Delta}_{[x_{in}, x]}) = \texttt{true} \right\}$$

where GC_t is the GC associated to transition t, and $S_{[x_{in}, x]}$ is the log state between x_{in} and x.

The interval $[x_{\text{in}}, x_{\text{in}} + \mu(t)]$ is called the *dynamic fulfillment interval* of the transition. It imposes the minimum and maximum timestamp in which GC_t will be evaluated to decide the firing of t. If $GC_t(S_{[x_{in}, x]}) = \texttt{false} \quad \forall x \in [x_{\text{in}}, x_{\text{in}} + \mu(t)]$ then ToC returns -1.

[3] K^* is the set of all possible sets of tokens in the net.

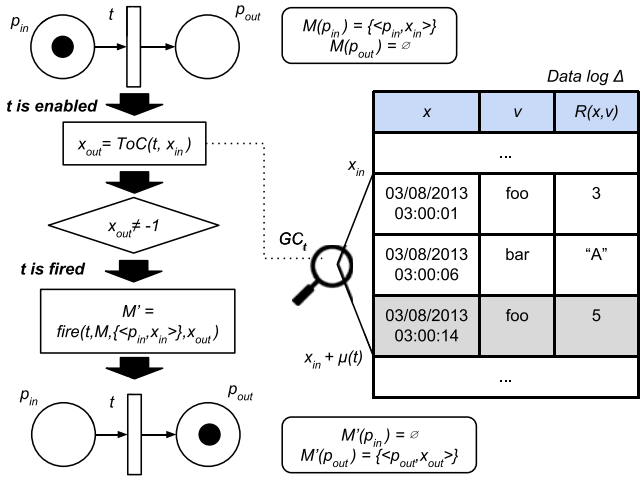

Fig. 3. General scheme for the dynamics of a WF-Net-based OP. In this example, the guard transition GC_t is associated to the procedural step "Supervise that `foo` is greater than 4." The entry of the data log that meets the condition for the first time (and gives value to x_{out}) is shaded.

Note that the situation may arise where two tokens $\langle p, x_1\rangle$ and $\langle p, x_2\rangle$ share the same place $p \in \bullet t$, but have different timestamps. As a consequence, we may have $ToC(t, x_1) \neq -1$ and $ToC(t, x_2) = -1$, i.e., only one of the tokens is *valid* for the transition guard. In this situation, a classical PN firing could choose the invalid token to be fired. Furthermore, during the transition firing, we want that the timestamp of the output token(s) is updated to the result of the function ToC, so that firings represent the concept of progress in the procedure following. For these reasons, the definition of a firing must be adapted for a WF-Net-based OP.

Definition 5: Let $t \in T$ be a transition of a given WF-Net-based OP, let $M \in M^*$ be a marking of that net, and let $x_{\text{out}} \in [0, \infty)$. Given a set of tokens $\kappa \in K^* \mid \forall k \in \kappa\, k.p \in \bullet p \wedge ToC(t, k.x) \neq -1$, a *firing* is a function $T \times M^* \times K^* \times [0, \infty) \to M$ that produces a marking $M' = \text{firing}(t, M, \kappa, x_{\text{out}})$ where

$$M'(p) = \begin{cases} M(p) \setminus \kappa & \text{if } p \in \bullet t \setminus t\bullet \\ M(p) \cup \{\langle p, x_{\text{out}}\rangle\} & \text{if } p \in t\bullet \setminus \bullet t \\ (M(p) \setminus \kappa) \cup \{\langle p, x_{\text{out}}\rangle\} & \text{if } p \in \bullet t \cap t\bullet \\ M(p) & \text{otherwise.} \end{cases}$$

The third case of the above definition covers the situations where a place is both an input and an output of the fired transition (loops). Note that the firing is associated to a set of tokens instead of a single one. This way, we cover the case of AND-Join transitions where there are more than one input places and thus, the firing must consume one token from each of them. Also note that, although we use the concept of time as part of the tokens, we are not violating the atomic property of transition firings from classical PNs, i.e., there is no time elapse between the enabling and the firing of a transition.

Fig. 3 represents a general scheme for the dynamics of a WF-Net-based OP with the concepts explained above. Once the transition t is enabled (i.e., there is at least one token in every input place of t), we compute $x_{out} = ToC(t, x_{\text{in}})$. If $x_{out} \neq -1$, the transition may fire, consuming the input token and producing the corresponding output tokens with a timestamp equal to x_{out} ($x_{out} \geq x_{in}$). Otherwise, the transition is not fired and the token remains in the input place, due to the log state does not meet the GC of the transition in the interval $[x_{\text{in}}, x_{\text{in}} + \mu(t)]$.

Given the definitions from Definitions 3 and 4, we can describe formally the dynamical behavior of every type of procedural step in an OP.

1) *Actions* comprise one transition which must be fired only if the action was performed within the associated maximum step duration (μ). If the action is found in the data log within that interval, the transition is fired, and the Time of Completion (ToC) establishes the timestamp of the output token to a value $x_{\text{out}} \in [x_{in}, x_{in} + \mu]$, indicating the moment in which the action was completed.
2) *Checks* comprise two transitions, one controlling the behavior of the OP in those cases that the condition to check has been fulfilled, and the other governing the contingency route for the cases where the condition has not been fulfilled (see Fig. 2). Let GC_t be the GC for the "`fulfilled`" transition, the corresponding $GC_{\bar{t}}$ for the "`!fulfilled`" transition is the complementary of GC_t, i.e., a function where $GC_{\bar{t}} = !GC_t$ for every input. As a consequence, those transitions become mutually exclusive, resulting in an XOR-Split block. Checks are considered to be instantaneous, thus $\mu = 0$ (the dynamic fulfillment interval becomes a point), and the ToCs for both transitions do not add time to the output tokens ($x_{out} = x_{in}$).
3) *Supervisions* combine two mutually exclusive transitions with complementary GCs as in the case of *Checks* (XOR-Split block). Tokens are evaluated in a dynamic interval $[x_{in}, x_{in} + \mu]$. It the supervision is fulfilled, the "`fulfilled`" transition will be fired, setting a new timestamp to the token, x_{out}, indicating the exact moment in which the *Supervision* was fulfilled. Otherwise, the "`!fulfilled`" transition will fire, and $x_{out} = x_{in}$, due to the conditions of the two transitions are complementary.
4) AND-Split and AND-Join transitions, also called *invisible* transitions due to they are not directly related to a procedural step, have a trivial GC, i.e., GC_t returns always `true`. Thus, they do not add time to the input tokens during the firing ($ToC(t, x_{in}) = x_{in}$).

Summarizing, based on the definitions provided in this section, a WF-Net-based OP is formally expressed as the tuple $(P, T, I, O, G, \mu, p_i, p_e)$, where $\mu : T \to [0, \infty)$ maps every transition to the associated maximum step duration, and $G : T \to GC^*$ does the same with the GCs [GC^* is the space of GCs (see Definition 3)].

One common issue to address when defining a workflow process is the formal verification of the *soundness* of the workflow. A sound workflow requires that at the moment it terminates there is a token in the end place and all the other places are empty. Since a WF-Net-based OP is aimed to be used as an evaluator, soundness is not desired, and instead, we want the

process to be able to finish in different markings so that different evaluations can be assigned based on the final marking. Formally, a WF-Net is *sound* if and only if the corresponding extended WF-Net, which connects the end place to the initial place, meets the following properties [19].

1) It is *live*, which means that for every reachable marking M' and every transition t, there is a marking M'' reachable from M' which enables t.
2) It is *bounded*, which means that for each place p there is a natural number n such that for every reachable state the number of tokens in p is less than n.

In a WF-Net-based OP, we might have one or many concurrent containers holding check or supervision substeps. In such cases, there is the possibility that some of them are fulfilled and some other not. Thus, the AND-join transition that concludes the `fulfilled` branch of the concurrent container would not be fired, leaving some tokens (the ones associated to the fulfilled substeps) blocked. Since in the extended net the end place is connected to the initial place, this in turn can result into a sink of tokens. Thus, such a WF-Net-based OP would not be bounded. Regarding the live property, it is easy to see that the condition of an action step can be fulfilled for a given token at a specific dynamic fulfillment interval, but when the token comes back from the end place to the initial place, it may have a higher timestamp, and the same condition may not be fulfilled, blocking the token and transforming some markings that were reachable before into unreachable markings. Thus, a WF-Net-based OP is not *live*, and hence it is not sound neither.

IV. WF-Net-Based Automatic Procedure Following Evaluation (WF-Net APFE)

Although the idea of modeling an OP has many advantages and applications, the OP dynamics defined in this paper are focused on the application of the model to analyze the procedure following level of an operator. To the best of our knowledge, this concept, which can be named as APFE, is new, and only a few works have issued the topic by other techniques, such as sequence alignment methods [20].

Algorithm 1 summarizes the process of Automatic Procedure Following Evaluation (APFE): given a WF-Net-based OP with all its attributes (OP), a data log (Δ) and a start time when we want to begin the evaluation (x_0), a single token $\langle OP.p_i, x_0\rangle$ is created in the initial place of the OP (recall that this place always exists in a WF-Net). This token comprises the initial marking of the OP, namely M. During the execution of the algorithm, we will need to keep track of the *invalid conditions*. An invalid condition is a pair $\langle t, k\rangle \in T \times K$, such that $ToC(t, k.x) = -1$, i.e., such that the token k does not meet the GC of the transition t. We denote by IC the set of tracked invalid conditions. At the beginning of the algorithm, IC is initialized to the empty set.

After the initialization, the dynamics of WF-Net-based OP are executed in a loop (see line 5) while the following conditions are met.

1) There is at least one enabled transition in the current marking of the net, i.e., $\exists t \in T \mid \forall p \in {\bullet t}\, M(p) \neq \emptyset$.

Algorithm 1: Workflow Net (WF-Net)-based Automatic Procedure Following Evaluation (WF-Net APFE).

Input: $OP = (P, T, I, O, C, \mu, p_i, p_e)$ is an Operating Procedure modelled as a Workflow Net (WF-Net). $\Delta = (V, U, R)$ is the data log of the operation. x_0 represents the start time for the evaluation.

Output: Tuple containing the following elements: A boolean value indicating whether the procedure has been successfully followed or not, the final marking of the Petri Net at the end of the process, and the time spent in the Procedure Following Evaluation.

1: **function** APFE (OP, Δ, x_0)
2: $M \leftarrow \{\langle OP.p_i, x_0\rangle\}$
3: $OP.setMarking(M)$
4: $IC \leftarrow \emptyset$ ▷ Invalid conditions
5: **while** ($\exists t \in enabledTransitions(OP) \mid \forall p \in {\bullet t}\, \exists k \in M(p) \mid \langle t, k\rangle \notin IC$) **do**
6: $\kappa \leftarrow \emptyset$ ▷ Valid tokens
7: $\gamma \leftarrow \emptyset$ ▷ Times of completion
8: **for all** $p \in {\bullet t}$ **do**
9: k $\leftarrow \arg\max_{k' \in M(p) \mid \langle k', t\rangle \notin IC} (k'.x)$
10: $\kappa \leftarrow \kappa \cup \{k\}$
11: $\gamma \leftarrow \gamma \cup \{\langle k, ToC(t, k.x)\rangle\}$ ▷ See Def. 4
12: **end for**
13: **if** $\nexists k \in \kappa \mid \langle k, -1\rangle \in \gamma$ **then**
14: $x_{out} \leftarrow \max_{\langle k,x\rangle \in \gamma} (x)$
15: $M' \leftarrow firing(t, M, \kappa, x_{out})$ ▷ See Def. 5
16: $M \leftarrow M'$
17: **else**
18: $IC \leftarrow IC \cup \{\langle t, k\rangle | k \in \kappa \mid \langle k, -1\rangle \in \gamma\}$
19: **end if**
20: **end while**
21: $M_f \leftarrow OP.getMarking()$
22: $time \leftarrow \max_{k \in M_f} (k.x - x_0)$
23: **if** $|M_f(OP.p_e)| > 0$ **then**
24: **return** $<$ `true`, $M_f, time >$
25: **else**
26: **return** $<$ `false`, $M_f, time >$
27: **end if**
28: **end function**

2) For every input place, there is at least one token that is not part of a tracked invalid condition, i.e., $\forall p \in {\bullet t}\, \exists k \in M(p) \mid \langle t, k\rangle \notin IC$.

For each iteration of the loop, the algorithm tries to fire a transition. A set of tokens to be consumed, namely $\kappa \subset M({\bullet t})$, will be created by adding, from each input place, the "valid" token with the maximum timestamp. This rule is applied in order to avoid the ambiguity of the process when selecting input tokens that share the same place. Then, the time of completion function (ToC) is computed for all the selected input tokens. The dynamic fulfillment interval $[x, x + \mu(t)]$ used in the ToC depends on the step type to which the transition belongs (see Fig. 2). The ToC will search in data log (Δ) whether the GC associated to the transition is fulfilled between x and $x + \mu(t)$,

and based on this it will decide both the firing of the transition and the timestamp of the output token(s). In case that the guard condition is met for every selected input token, i.e., in case that the times of completion for all of them are be different from -1, the transition will be fired, consuming the tokens from κ and producing the output tokens properly. The timestamp of the output tokens (x_{out}) will be set as the maximum ToC. Again, this rule is applied in order to be deterministic in the cases where $|\kappa| > 1$, which is something that, in the context of a WF-Net-based OP, only happens at an AND-Join transition at the end of concurrent containers. Here, the input tokens represent the response of the operator in each of the branches of the container, and thus, keeping the maximum timestamp among them is the correct way to track the general procedure following time. In case that some of the selected input tokens $k \in \kappa$ do not meet the GC of the transition, the corresponding pairs $\langle k, t \rangle$ are added to the set of invalid conditions IC.

When the above loop ends, the net reaches a marking M_f in which no transition can be fired (Note that a transition can be enabled but not fired). The maximum timestamp of a token in M_f will allow us to compute the time spent in the procedure following (see line 22 of the Algorithm 1), and the rest of the information contained in M_f will be used to judge whether the OP has been successfully completed or not.

1) If M_f contains a token placed in the end place (p_e) of the net (see Algorithm 1, line 23), then we consider that the operator has followed the OP successfully.
2) Otherwise, if the initial token has been locked in some intermediate place of the net, it means that some procedural step in the OP has not been performed correctly. Nevertheless, the algorithm returns the information of the marking M_f so that we can analze which steps are causing troubles for the operators.

V. Case Study: Emergency OP for the "Engine Bay Overheating" Alert in an Unmanned Aircraft System (UAS)

The use of UASs and their related technologies is becoming a hot topic in the last few years, from both the research and the industrial interests. From an industrial perspective, these technologies have proven to provide different complex applications in real and heterogeneous domains, such as coastal monitoring, traffic management, and agriculture among others [21]. From the research point of view, UAS operations have been highly studied for many years in the field of aeronautics and cooperative control [22], though lately more and more computer science fields are emerging on the scene, by developing complex algorithms for mission planning [23] and for extracting behavioral patterns among operators [24], [25], to mention a few examples.

Typically, the whole system involved in an operation with UAV is known as Unmanned Aircraft System (UAS), and it is mainly composed of the UAV itself and a ground control station (GCS), from where a single operator (or a crew) remotely supervises and controls the route and the advances of the UAV in a specific mission. Due to the high costs involved in any mission of this kind, every critical step or possible failure is controlled by the guidelines of detailed action checklists, as it happens in manned operations.

ALERT NAME
ENGINE BAY OVERHEATING (EBO)

DESCRIPTION
Risk of damage or loss of equipment in engine bay.

DO-LIST OPERATING PROCEDURE

1. In Main Control Panel, check:
   - Engine Temperature Panel is high-yellow
   - Engine Temperature is over 70ºC
2. Modify UAV altitude by using ALTITUDE command
3. Supervise that EBO caution disappears from the alert panel
4. IF EBO caution persists:
   4.1. Command GO-WP to waypoint IAF
   4.2. Command LANDING
5. If EBO caution disappears:
   - Resume Mission by changing the control mode to AUTOMATIC

Fig. 4. Description of the OP for the "Engine Bay Overheating" alert, as described in an Airbus-based checklist document for UAV operations.[4]

In order to put into practice, the theoretical aspects described in this paper, we will study how to model and evaluate the response of an operator in an UAS while following an OP designed to face the alert "Engine Bay Overheating." This alert is fired when the temperature inside the engine bay of an Unmanned Aerial Vehicle (UAV) is over the equipment operative limit. The risk of damage or loss of equipment creates a serious danger, and thus, in case the operator is unable to reduce the temperature, he/she must act immediately aborting the mission and landing the UAV.

A. Creating the WF-Net for the "Engine Bay Overheating" OP

In Fig. 4, a snapshot of an OP document related to the alert "Engine Bay Overheating" is shown. There are some important aspects to consider in this document.

1) The different checks comprising the first procedural step are carried out in parallel. These checks are considered to be performed visually and instantaneously.
2) If any of the checks comprising the first procedural step is not fulfilled, it is considered that the alert is not real, i.e., it is a spurious, and the operator should not continue the next steps of the OP.

Fig. 5 shows a graphical representation of the WF-Net that results from applying the modeling techniques proposed in this paper to the "Engine Bay Overheating" OP shown in Fig. 4. The type and maximum step duration (μ) for every step is given in Table I. Based on this, each procedural step has been expressed as a set of places, transitions and arcs, following the

[4]The contents of the OP shown here have been adapted from a real checklist used by Airbus Defence and Space Company. However, and due to a confidential agreement with this company, any detail or sensitive information has been removed.

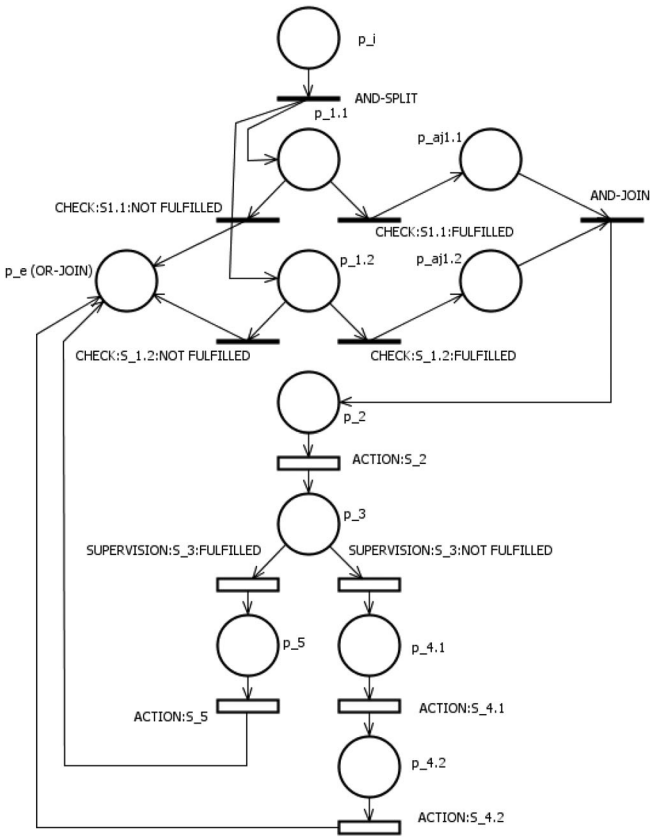

Fig. 5. Petri Net-based workflow model for the "Engine Bay Overheating" OP showed in Fig. 4.

rules summarized in Fig. 2. To allow a better understanding of the model, transitions are labeled according to the name of the step to which they belong, prefixed by the step type (e.g., `CHECK:S_1.2`) and suffixed by the labels "`FULFILLED`" or "`NOT FULFILLED`" for *check* and *supervision* steps. Below are remarked some details related to the modeling process.

1) Step 1 is modeled as concurrent container, due to the parallel nature of the substeps contained in it. Thus, an AND-Split/AND-Join routing block is added and linked to the `fulfilled` transition of every substep, ensuring the fulfillment of all of them to continue. Likewise, the place p_e acts not only as an end place, but also as an OR-Join routing block, since it gathers the nonfulfillment of any of the substeps.
2) In Step 2, it is mandatory to follow the order of the substeps, thus it is modeled as a sequential container.
3) Step 3: "Supervise that EBO caution disappears ..." makes the operator wait for a certain period of time until the condition is fulfilled. This is modeled as a supervision step, whose associated maximum step duration (μ) establishes the maximum waiting time.

The data logs used in this experimentation has been extracted using the message formats and variables described in the *NATO Standardization Agreement 4586* (STANAG 4586) [26], that record the state of the UAV and the GCS periodically during a mission. The data log will be accessed by the GCs of every transition in the model, in order to route the token throughout the workflow. The formal definition of the GCs in terms of entries of the data log is shown in Table I.

B. Testing the APFE Algorithm in the "Engine Bay Overheating" Model

In order to test the model created in the previous section and to make a better understanding about how the APFE algorithm (shown in Section IV) actually works in practice, some *test cases* have been developed. These test cases have been designed by modifying not only the conditions in which the alert "Engine Bay Overheating" is triggered, but also looking for different responses from the operators.

Table II shows the different test scenarios defined for this case study. Each of them simulates a possible combination of Fulfillment (F) and not Fulfillment (!F) of the different checks and supervisions found in the Engine Bay Overheating OP, in order to cover every possible way of facing the alert. The most interesting scenarios in terms of evaluating the operator response are TS1 and TS5, due to they are the only scenarios where the two initial check steps (Steps 1.1 and 1.2) are fulfilled, and thus, the entire Step 1 is completed and we can consider that the alert is *real*, thus the operator needs to continue the procedure. The rest of the cases represent *spurious*, i.e., false alerts.

On the other hand, the actions performed by a battery of test operators in order to face the alert are recorded in Table III. Each cell represents the time, measured in seconds since the beginning of the alert, when the operator completed the action, and in case the operator did not perform the action a "-" symbol is shown.

With regard to the parameter tuning involved in this case study, the start time for the APFE algorithm, x_0, must be set. For the sake of simplicity, it is set to the time when the alert was triggered. Regarding the implementation details: the design of the model shown in Fig. 4; the coding of the dynamics and the GCs of every transition; and the APFE algorithm itself, have been developed in Java 8, based on the PN simulation engine *PetriNetSim*.[5] The reason for using this tool is that it provides code generation capabilities, thus we can draw a WF-Net and then translate it into a Java class, where it is easier to implement the GCs shown in Table I in terms of a data log. This way, the WF-Net becomes part of a bigger evaluation tool, which fits our needs since we are not interested in simulating the WF-Net, but in using it externally to evaluate the operator response. The source code is available on Github.[6]

The results of running the APFE algorithm for each test case are shown in Table IV. There are a total of 80 test cases, combining the test scenarios shown in Table II with the test operators of Table III. Based on these results, we can extract some remarkable conclusions about this case study.

1) Regarding the test cases run in a *real-alert* test scenario (i.e., TS1 and TS5), only one test operator (TO8 in the case of TS1, and TO10 in TS5) was well evaluated by the

[5]PetrinetSim: https://github.com/zamzam/PetriNetSim.
[6]Source code of this work: https://github.com/lordbitin/TII-2017.

TABLE I
INFORMATION FOR MODELLING THE OP "ENGINE BAY OVERHEATING"

#	Step Type	μ	Guard Condition
Step 1	Concurrent Container	–	–
Step 1.1	Check	0	$GC(\epsilon) = \begin{cases} \texttt{true} & \text{if } \exists x \in [0,\infty) \mid \langle x, 58501.07, 0\rangle \in \epsilon \wedge \langle x, 58501.15, 0\rangle \in \epsilon \\ \texttt{false} & \text{otherwise} \end{cases}$
Step 1.2	Check	0	$GC(\epsilon) = \begin{cases} \texttt{true} & \text{if } \exists x \in [0,\infty), d \in \mathbb{R} \mid d \geq 70 \wedge \langle x, 55253.02, d\rangle \in \epsilon \\ \texttt{false} & \text{otherwise} \end{cases}$
Step 2	Action	15	$GC(\epsilon) = \begin{cases} \texttt{true} & \text{if } \exists \langle x, 2002.43.05, d\rangle, \langle x', 2002.43.05, d'\rangle \in \epsilon \mid d \neq d' \\ \texttt{false} & \text{otherwise} \end{cases}$
Step 3	Supervion	10	$GC(\epsilon) = \begin{cases} \texttt{true} & \text{if } \exists x \in [0,\infty) \mid \langle x, 55254.04, 3\rangle \in \epsilon \\ \texttt{false} & \text{otherwise} \end{cases}$
Step 4	Sequential Container	–	–
Step 4.1	Action	15	$GC(\epsilon) = \begin{cases} \texttt{true} & \text{if } \exists x \in [0,\infty) \mid \langle x, 2002.43.15, 4\rangle \in \epsilon \wedge \langle x, 2001.42.04, 33\rangle \in \epsilon \\ \texttt{false} & \text{otherwise} \end{cases}$
Step 4.2	Action	15	$GC(\epsilon) = \begin{cases} \texttt{true} & \text{if } \exists x \in [0,\infty) \mid \langle x, 2001.42.04, 19\rangle \in \epsilon \\ \texttt{false} & \text{otherwise} \end{cases}$
Step 5	Action	15	$GC(\epsilon) = \begin{cases} \texttt{true} & \text{if } \exists x \in [0,\infty) \mid \langle x, 2001.42.04, 33\rangle \in \epsilon \\ \texttt{false} & \text{otherwise} \end{cases}$

The variable names follow the format [Message type].[Field number] coming from STANAG 4586 (NATO Standardization Agreement 4586). All GCs are expressed in terms of a set of log entries $\epsilon \in E^*$ ($E = (0,\infty) \times V \times D$) from a given data log $\Delta = (V, U, R)$.

TABLE II
TEST SCENARIOS DEFINED FOR THIS CASE STUDY, WHERE F MEANS "`fulfilled`" AND $!F$, "`!fulfilled`"

	TS1	TS2	TS3	TS4	TS5	TS6	TS7	TS8
Step 1.1	F	!F	F	!F	F	!F	F	!F
Step 1.2	F	F	!F	!F	F	F	!F	!F
Step 3	F	F	F	F	!F	!F	!F	!F

TABLE III
TEST OPERATORS DEFINED FOR THIS CASE STUDY

	Action 2	Action 4.1	Action 4.2	Action 5
TO1	–	–	–	–
TO2	10	–	–	–
TO3	–	10	–	–
TO4	–	–	10	–
TO5	–	–	–	15
TO6	10	20	–	–
TO7	10	–	20	–
TO8	10	–	–	25
TO9	–	10	20	–
TO10	10	20	30	–

Each cell contains the ToC of an action. The symbol "-" means that the action has not been performed by the operator.

algorithm, which makes sense due to they are the only operators that performed all the necessary actions for that scenario. However, since the algorithm also returns the last marking of the PN, we can analyze which is the first missing action that causes an operator to fail in the procedure following. As an example, for the operators TO1,TO3,TO4,TO5, and TO9, the algorithm returns $1p_2$ as last state, which means that they are obviously missing action two of the procedure. That information could be used by an instructor to improve the performance of these operators during a training session.

2) For every test case run in a *spurious* test scenario (i.e., TS2, TS3, TS4, TS6, TS7, and TS8), the APFE algorithm returns `TRUE` as a general response and 0 as the time spent to complete the OP, regardless the test operator. This means that no matter what actions the operator perform, he/she will always be well evaluated with respect to this alert if the alert is not real. Although this can be a problem of the OP itself, which should ask for an explicit "acknowledge" action as a last step, our intention for the future is to improve the APFE algorithm so that it can evaluate when the operator "overreacts" to an alert with unnecessary actions.

VI. RELATED WORK

Once the proposed approach has been described and applied in a case study, it is important to provide a comparison study with previous works in the context of PFE. From a general perspective, there is a family of process mining techniques, called *conformance checking*, to compare the behavior between a business process model and an event log of the same process, which are comparable to this approach [5], [6]. Additionally, if we focus specifically on procedural behavior OPs, the main contributions devoted to evaluate it are found in [4] and [20]. Finally, the use of model checking and WfM for analyzing Emergency OPs [27], [28] is closely related to this work, but tackling the problem from the point of view of the system itself, not from the operator who acts externally with it. Thus, we will not include them into this study.

In order to perform the comparison, we consider multiple features.

TABLE IV
RESULTS OF THE APFE ALGORITHM FOR EACH TEST CASE (TEST SCENARIO + TEST OPERATOR) DEVELOPED IN THIS CASE STUDY

	TS1	TS2	TS3	TS4	TS5	TS6	TS7	TS8
TO1	$f, 1p_2, 0$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$	$f, 1p_2, 0$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$
TO2	$f, 1p_5, 15$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$	$f, 1p_{4.1}, 10$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$
TO3	$f, 1p_2, 0$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$	$f, 1p_2, 0$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$
TO4	$f, 1p_2, 0$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$	$f, 1p_2, 0$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$
TO5	$f, 1p_2, 0$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$	$f, 1p_2, 0$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$
TO6	$f, 1p_5, 15$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$	$f, 1p_{4.2}, 20$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$
TO7	$f, 1p_5, 15$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$	$f, 1p_{4.1}, 10$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$
TO8	$\mathbf{t, 1p_e, 25}$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$	$f, 1p_{4.1}, 10,$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$
TO9	$f, 1p_2, 0$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$	$f, 1p_2, 0$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$
TO10	$f, 1p_5, 15$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$	$\mathbf{t, 1p_e, 30}$	$t, 1p_e + 1p_{aj1.2}, 0$	$t, 1p_e + 1p_{aj1.1}, 0$	$t, 2p_e, 0$

Bolded cells mark the cases where the test scenario represents a real alert and the operator followed the procedure successfully. t and f denote `true` and `false`, respectively.

TABLE V
FEATURE COMPARISON BETWEEN PFE APPROACHES

	Auto.	MS	TS	ST	μ
EBAT [4]	✗	√	✗	All	✗
Sequence alignment [20]	√	✗	✗	A-C	✗
Conformance Checking [5], [6]	√	√	✗	A-C	✗
Proposed	√	√	√	All	√

1) *Automatic (Auto.)*: the approach can be executed automatically by a computer.
2) *Multiscenario*: the approach takes into account the different scenarios in which the same procedure can be evaluated.
3) *Time spent (TS)*: the approach records the time spent by the operator in the procedure following.
4) *Step types (ST)*: the approach accepts OPs with Actions ("A"), Checks ("C"), Supervisions ("S"), or all of them ("All").
5) *Maximum step duration* (μ): the approach allows us to constraint the maximum time spent in every procedural step.

A summary of the qualitative comparison based on these features is shown in Table V.

1) Dietz *et al.* describe in [4] an adaptation of the Event-Based Approach for Training (EBAT) framework to evaluate the procedural behavior in UASs. Here, an alert in the system is seen as an "event" and the procedural steps are specified as "targeted responses" for that event. The instructor is responsible for *manually* marking `YES/NO/unnecessary` for each targeted response, depending on the response of the operator and the current training scenario. No timestamps and maximum step duration are considered.
2) Kim describes in [20] an *automatic* quantification of the procedure following level based on sequence alignment techniques. It needs a reference (or "standard") sequence to compute the similarity between the operator response and that reference, so we would need to create multiple references to make a *multiscenario* evaluation, which is not scalable. Furthermore, it only takes into account the order of the operator actions, not the time spent in each of them.
3) Aalst *et al.* [5] and Garcia-Banuelos *et al.* [6] aim at the automatic detection of inconsistencies between a process model and its corresponding event log. The process model is also based on WF-Nets, which allows multiple execution contexts. Only actions and checks steps are allowed since they can be expressed as events. In these cases, the event log do not include timestamps to keep track of the conformance time or to constraint the maximum step duration.
4) Our approach describes the procedural behavior as a PN-based workflow, allowing an *automatic* evaluation of the *multiple scenarios* and routes of the procedure, and keeping track of the *time spent* by the operator in each step. Different step types can be modeled due to the flexibility of the GCs, and a maximum step duration can be set for every step.

VII. CONCLUSION

This paper has provided a new approach for modeling OPs based on PN-based Workflows, or WF-Nets, focusing on the use of the model as a tool to allow an APFE. No restriction is imposed over the domain field in which this approach could be used. Since the dynamics of a WF-Net-based operating procedure (OP) are based on the presence of a data log, we need that the system in which the OP is executed provides *data-recording* capabilities in some way. In other words, the concepts of OP models and APFE are closely linked to the data recorded by the system.

In order to test the proposed modeling methods and the APFE algorithm, an OP from the area of UAV operations has been modeled, in concrete, the do-list associated to the alert *"Engine Bay Overheating."* Some test cases have been designed to demonstrate the usefulness of the model and to show how the APFE algorithm works under different scenarios and operators.

Several future works and extensions are currently under consideration.

1) In order to allow an easy creation of WF-Net-based OPs, a Domain-Specific Language tool for translating the concepts and relations defined for an OP to the corresponding elements of a WF-Net would be very useful, especially for those domain experts who need to model large number of OPs in a system.

2) The current APFE algorithm described in this paper could be enhanced by returning all the missing actions and overreactions of the operator.
3) Finally, in order to test the generality of the proposed method, more case studies are needed, comparing models among different domain areas (aviation, healthcare, etc.) in which the shape of the OPs would be different.

Acknowledgements

The authors would like to acknowledge the support from Airbus Defence & Space, specially from Savier Open Innovation project members: José Insenser, Gemma Blasco, Juan Antonio Henríquez and César Castro. Finally, we would like to thank the reviewers and the Editor in Chief for the different suggestions and comments made to this work.

References

[1] J. P. Ornato and M. A. Peberdy, "Applying lessons for commercial aviation safety and operations to resuscitation," *Resuscitation*, vol. 85, no. 2, pp. 173–176, 2014.
[2] D. R. Urbach, A. Govindarajan, R. Saskin, A. S. Wilton, and N. N. Baxter, "Introduction of surgical safety checklists," *J. Med.*, vol. 370, no. 11, pp. 1029–1038, 2014.
[3] R. Kelly, V. Monnelly, and B. Stenson, "G566 (p) checklists for time-critical equipment failure during patient transport," *Arch. Disease Childhood*, vol. 100, pp. A255–A255, 2015.
[4] A. S. Dietz, J. R. Keebler, R. Lyons, E. Salas, and V. C. Ramesh, "Developing unmanned aerial system training: An event-based approach," in *Proc. Human Factors Ergonom. Soc. Annu. Meeting*, 2013, vol. 57, pp. 1259–1262.
[5] W. Van der Aalst, A. Adriansyah, and B. van Dongen, "Replaying history on process models for conformance checking and performance analysis," *Data Mining Knowl. Discovery*, vol. 2, no. 2, pp. 182–192, 2012.
[6] L. Garcia-Banuelos, N. van Beest, M. Dumas, M. La Rosa, and W. Mertens, "Complete and interpretable conformance checking of business processes," *IEEE Trans. Softw. Eng.*, vol. PP, no. 99, pp. 1–29, 2017.
[7] A. F. Arriaga *et al.*, "Simulation-based trial of surgical-crisis checklists," *New England J. Med.*, vol. 368, no. 3, pp. 246–253, 2013.
[8] H. S. Kramer and F. A. Drews, "Checking the lists: A systematic review of electronic checklist use in health care," *J. Biomed. Informat.*, vol. 71, pp. S6–S12, 2017.
[9] H. A. Reijers, H. Leopold, and J. Recker, "Towards a science of checklists," *Proc. 50th Hawaii Int. Conf. Syst. Sci.*, 2017, pp. 5773–5782.
[10] C. Thongprayoon, A. M. Harrison, J. C. O'Horo, R. A. S. Berrios, B. W. Pickering, and V. Herasevich, "The effect of an electronic checklist on critical care provider workload, errors, and performance," *J. Intensive Care Med.*, vol. 31, no. 3, pp. 205–212, 2016.
[11] A. Ikram, A. Anjum, and N. Bessis, "A cloud resource management model for the creation and orchestration of social communities," *Simul. Model. Pract. Theory*, vol. 50, pp. 130–150, 2015.
[12] A. Pla, P. Gay, J. Meléndez, and B. López, "Petri net-based process monitoring: a workflow management system for process modelling and monitoring," *J. Intell. Manuf.*, vol. 25, no. 3, pp. 539–554, 2014.
[13] K. Jensen, *Coloured Petri Nets: Basic Concepts, Analysis Methods and Practical Use*, vol. 1. New York, NY, USA: Springer, 2013.
[14] J. Liu, E. Pacitti, P. Valduriez, and M. Mattoso, "A survey of data-intensive scientific workflow management," *J. Grid Comput.*, vol. 13, no. 4, pp. 457–493, 2015.
[15] W. Liu, Y. Du, M. Zhou, and C. Yan, "Transformation of logical workflow nets," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 44, no. 10, pp. 1401–1412, Oct. 2014.
[16] A. Arkoma, M. Hänninen, K. Rantamäki, J. Kurki, and A. Hämäläinen, "Statistical analysis of fuel failures in large break loss-of-coolant accident (LBLOCA) in EPR type nuclear power plant," *Nuclear Eng. Desi.*, vol. 285, pp. 1–14, 2014.
[17] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proc. ACM SIGOPS 22Nd Symp. Oper. Syst. Principles*, 2009, pp. 117132.
[18] M. de Leoni and W. M. P. van der Aalst, "Data-aware process mining: Discovering decisions in processes using alignments," in *Proc. 28th Annu. ACM Symp. Appl. Comput.*, 2013, pp. 1454–1461.
[19] R. Klimek, "A system for deduction-based formal verification of workflow-oriented software models," *Int. J. Appl. Math. Comput. Sci.*, vol. 24, no. 4, pp. 941–956, 2014.
[20] Y. Kim, "Analyzing procedural behaviors of human-machine systems using sequence alignment techniques," *Jpn. J. Ergonom.*, vol. 49, no. Suppl., pp. S451–S454, 2013.
[21] I. Colomina and P. Molina, "Unmanned aerial systems for photogrammetry and remote sensing: A review," *ISPRS J. Photogramm. Remote Sens.*, vol. 92, pp. 79–97, 2014.
[22] J. Ruiz, A. Viguria, J. Martinez-de Dios, and A. Ollero, "Immersive displays for building spatial knowledge in multi-UAV operations," in *Proc. 2015 IEEE Int. Conf. Unmanned Aircr. Syst.*, 2015, pp. 1043–1048.
[23] C. Ramirez-Atencia, S. Mostaghim, and D. Camacho, "A knee point based evolutionary multi-objective optimization for mission planning problems," in *Proc. Genetic Evol. Comput. Conf.*, 2017, pp. 1216–1223.
[24] V. Rodríguez-Fernández, A. Gonzalez-Pardo, and D. Camacho, "Modelling behaviour in UAV operations using higher order double chain Markov models," *IEEE Comput. Intell. Mag.*, vol. 12, no. 4, pp. 28–37, 2017.
[25] V. Rodríguez-Fernández, H. D. Menéndez, and D. Camacho, "Analysing temporal performance profiles of UAV operators using time series clustering," *Expert Syst. Appl.*, vol. 70, no. 15, pp. 103–118, 2017.
[26] S. Frazzetta and M. Pacino, "A STANAG 4586 oriented approach to UAS navigation," *J. Intell. Robot. Syst.*, vol. 69, no. 14, pp. 21–31, 2013.
[27] C. Liu, Q. Zeng, H. Duan, M. Zhou, F. Lu, and J. Cheng, "E-Net modeling and analysis of emergency response processes constrained by resources and uncertain durations," *IEEE Trans. Syst. Man, Cybern. Syst.*, vol. 45, no. 1, pp. 84–96, Jan. 2015.
[28] W. Tan and M. Zhou, *Business and Scientific Workflows: A Web Service-Oriented Approach*, vol. 5. Hoboken, NJ, USA: Wiley, 2013.

Víctor Rodríguez-Fernández received the B.Sc. degree in computer science and the B.Sc. degree in mathematics from the Universidad Autónoma de Madrid, Madrid, Spain, in 2013, and the M.Sc. degree in computer science from the Universidad Autónoma de Madrid in 2015. He is currently working toward the Ph.D. degree at Escuela Politécnica Superior (EPS-UAM), Madrid, Spain.

He is currently involved with the AIDA research group at EPS-UAM. His research interests include conformance checking, behavioral modeling, pattern recognition, and unmanned aerial systems.

Antonio Gonzalez-Pardo (M'15) received the Ph.D. degree in computer science from the Universidad Autónoma de Madrid, Madrid, Spain, in 2014.

He is currently a Lecturer with the Universidad Autónoma de Madrid. His main research interests include computational intelligence (genetic algorithms, PSO, SWARM intelligence, etc.), and machine learning techniques. The application domains for his research are constraint satisfaction problems, complex graph-based problems, optimization problems, and video games.

David Camacho (M'14) received the Ph.D. degree in computer science from the Universidad Carlos III de Madrid, Madrid, Spain, in 2001.

He is currently an Associate Professor with the Department of Computer Science, Universidad Autónoma de Madrid, Spain, and the Head of the Applied Intelligence and Data Analysis Group. He has authored or coauthored more than 200 journals, books, and conference papers. His research interests includes data mining (clustering), evolutionary computation (GA and GP), multiagent systems and swarm intelligence (Ant Colonies), automated planning and machine learning, or video games among others.